Let's talk about the function `create_pytorch_dataloader`, as this is where the DeltaTorch loader is used. This is the main differentiator from classical PyTorch code. `Create_pytorch_loader` is a function to create a PyTorch DataLoader, which can be used directly during training. During training, this function checks if the Delta Table with a specified path exists, if the provided columns exist under the schema of a table, and if the table is not empty. It also performs a shuffle function under the hood, so you do not need to take care of it (which is why the `id_field` is required; it's used for sharding and parallelization of loading). As with the classic PyTorch DataLoader, you can apply transformations to your batch of images, modify the batch size, modify the number of workers that are used to load the data, and choose whether you want to apply the shuffle or not. We have also provided an example with a classical PyTorch Dataloader inside the `Ch6-01` notebook under the header "Native Torch Loader" in case you would like to store your images in a volume instead.

Now let's move on to the model! *Figure 6.28* shows the code in cell 7, though we have minimized some functions for easier reading. The main difference between a classic PyTorch code and the one you see below are the x and y definitions from the batch.

```
class LitCVNet(pl.LightningModule):

    def __init__(self, num_classes = 6, learning_rate= 0.0001, family = "resnext", momentum = 0.1,  dropout_rate = 0):…

    def get_model(self, num_classes):
        """
        This is the function that initialises our model.
        If we wanted to use other prebuilt model libraries like timm we would put that model here
        """
        backbone = torchvision.models.wide_resnet50_2(pretrained=True)
        for param in backbone.parameters():
            param.required_grad = False
        num_ftrt = backbone.fc.in_features
        backbone.fc = nn.Linear(num_ftrt, num_classes)
        return backbone

    # We do not overwrite our forward pass
    def forward(self, x):…

    def training_step(self, batch, batch_idx):
        x = batch["content"]
        y = batch["label_id"]
        logits = self.forward(x)
        loss = self.loss(logits,y)
        # Track accuracy
        acc = self.accuracy(logits, y)
        # required format
        self.log("loss", torch.tensor([loss]), on_step=True, on_epoch=True, logger=True)
        self.log("acc", torch.tensor([acc]), on_step=True, on_epoch=True, logger=True)
        return  {"loss": loss, "acc": acc}

    def validation_step(self, batch, batch_idx):…

    def test_step(self, batch, batch_idx):…

    # predict_step is optional unless you are doing some predictions
    def predict_step(self, batch, batch_idx, dataloader_idx=0):…

    def configure_optimizers(self):…
```

Figure 6.28 – Function for initializing and training model